Vemu Library Management System

PRODUCT REQUIREMENTS DOCUMENT (PRD)

Library Management System (VEMU)

Product Overview

The **Library Management System** is a web-based application designed to manage library operations such as book tracking, user management, issuing/returning books, and notifications.

It provides:

- Admin control over books and users
- Student/user access to browse and request books
- Automated email notifications
- Record tracking for library activities

Backend

In this web application backend was totally done by using node.js and MonogDB for database.

First of all I install npm init command is used to initialize a new Node.js project by creating a package.json file in your project's directory. This file acts as the Heart of your project, Storing data and managing dependences.

After completing node.js installation if you not created a file just create a file index.js in side the backend why should I specially tell create index.js file means while we are installing node.js without creating file that node.js create default file name is index.js inside the package.json.

After completing the backend setup we need to install some modules like express, nodemon, cors, bcrypt, mongoose nodemailer and dotenv.

How install that modules

First of all we should open terminal and check if user terminal is open with backend folder are not if terminal was not opened with backend folder you

should be enter like this in terminal **cd backend** (or) **backend_foldername** that should open with backend folder**.**

Else open like this click on your file like indes.js and in right side we see open with integrated terminal click on it will open with your backend folder.

Installing and uses of express

After open the terminal enter like this for install node modules **npm install express**.

```
1015        });
1016
1017        res.json(results);
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\leela\OneDrive\Desktop\Vemu library management system\backend> npm install express

up to date, audited 127 packages in 3s

32 packages are looking for funding
  run `npm fund` for details

4 vulnerabilities (1 low, 1 moderate, 2 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
PS C:\Users\leela\OneDrive\Desktop\Vemu library management system\backend>
```

Epress.js is a minimal and flexible Node.js web application framework used to build server-side application and API's. It simplifies the complex, low-level HTTP code of Node.js into a more manageable and organized structure.

Completing the installing the express we import that should be in the index.js file after importing the express.js we should be assign that express to like app varable or another varable.

Installing and uses of nodemon

After completing the installing express we should install nodemon. Same nodemon also install like that only in terminal **npm install nodemon**. Nodemon is a auto-restart your application without restart the node.js manually after updating the the code. After installed nodemon we don't want to import that into the file.

Installing and uses of cors

Completing installing the nodemon we should install cors like same **npm install cors.** cors is a cors-origin resource sharing it handles the middleware packages

from the npm. Without cors, browser block cross origin requests by default due to the same origin policy.

After completing installing cors we import into the file. And enable that cors like this **app.use(cors());**

Installing and uses of bcrypt

Install bcrypt same as **npm install bcrypt.** Import that bcrypt into the file it is used to securely hashing and storing user passwors. It protect credentials by converting plane-text password into unreadable hashes that are resistant to brute-force and rainbow table attacks.

Installing and uses of MongoDB

Install MongoDB as npm install mongoose and import it into the file. MongoDb is a popular NoSQL document-oriented database used for high-volume data storage, Modern application, and real-time analystics, storing data in flexible JSON-like BSON documents.

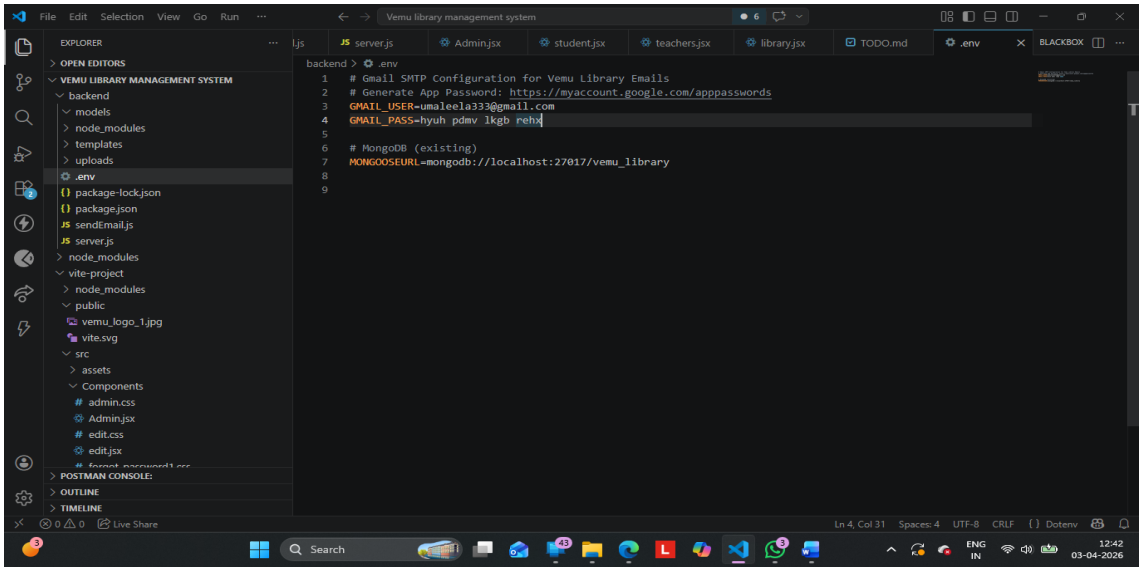
Installing and uses of nodemailer

Install nodemailer as npm install nodemailer and import it into the file. Nodemailer is popular node.js module used to send emails from the backend application, supporting SMTP, OAuth2, and various transport services like AWS, SES.

Installing and uses of dotenv

Install .env as npm install dotenv and import it into the file and create .env file. It is securely separate configuration and security such as API keys, database URL, and port numbers from the source code. Enchancing security and facilitating easier deployment across development, staging, and production

environment.



Implementation process

After completing installation of modules we need run the server. Like this

```
        res.json(results);
    } catch (error) {
        console.error('History search error:', error);
        res.status(500).json({ message: 'Search failed' });
    }
});

app.listen(3000, () => {
    console.log("Server running on 3000");
});
```

After that we need to connect mongoose for store user details and booking details, etc…

In the below image you can see how to connect the MongoDB to the application.

```
mongoose.connect(process.env.mongooseURL || 'mongodb://localhost:27017/vemu_library')
    .then(() => console.log("MongoDB connected successfully"))
    .catch(err => {
        console.error("MongoDB connection error:", err);
        console.log("Continuing without DB - static data mode");
    });
```

In the above image we seen how to connect and run the mongoDB. Now we can see how to store the into the database. First of all we need create a some schemas. In this schema each document consists of field-value pairs and can include various data types like strings, numbers, arrays, and nested documents. This structure allows for storing complex data in a single record.

```
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

// Email configuration moved to sendEmail.js


// Student Schema
const userSchema = new mongoose.Schema({
    username: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    college_id: { type: String, required: true },
    department: { type: String, required: true },
    mobileNumber: { type: String },
    password: { type: String, required: true }
});
const User = mongoose.model("User", userSchema);

// Teacher Schema
const teacherSchema = new mongoose.Schema({
    username: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    college_id: { type: String, required: true },
    department: { type: String, required: true },
    mobileNumber: { type: String },
    password: { type: String, required: true }
});
const Teacher = mongoose.model("Teacher", teacherSchema);

// Admin Schema - FIXED
const adminSchema = new mongoose.Schema({
    username: { type: String, required: true },
```

Using CRUD Operations to store, fetch, update, delete the user details

use of app.post method in the projects

The app.post() method is used in projects to handle HTTP POST requests to a s pecified route. It is commonly used for submitting forms or sending data to a se rver from clients. Here are some key uses of the app.post() method in.

In the below code I used bcrypt to hashed the password.

```
// Admin login - FIXED path to '/admin_login'
app.post('/admin_login', async (req, res) => {
    try {
        const { email, password } = req.body;
        const user = await Admin.findOne({ email });
        if (!user) {
            return res.status(404).json({ message: "User not found" });
        }
        const isMatch = await bcrypt.compare(password, user.password);
        if (isMatch) {
            res.status(200).json({
                message: 'Admin login successful',
                user: {
                    username: user.username,
                    email: user.email,
                    college_id: user.college_id,
                    mobileNumber: user.mobileNumber || ''
                }
            });
        } else {
            res.status(400).json({ message: "Invalid Email or Password" });
        }
    } catch (err) {
        console.error('Admin Login Error:', err);
        res.status(500).json({ message: 'Server Error Occurred' });
    }
});

// Library login - FIXED destructuring and bcrypt
app.post('/library_login', async (req, res) => {
```

Login page connection in this login page also we used bcrypt to compare the password.

```
// Student login
app.post('/student_login', async (req, res) => {
    try {
        const { email, password } = req.body;
        const user = await User.findOne({ email });
        if (!user) {
            return res.status(400).json({ message: 'Invalid Email or Password' });
        }
        const isMatch = await bcrypt.compare(password, user.password);
        if (isMatch) {
            res.status(200).json({
                message: 'Login successful',
                user: {
                    username: user.username,
                    email: user.email,
                    college_id: user.college_id,
                    department: user.department,
                    mobileNumber: user.mobileNumber
                }
            });
        } else {
            res.status(400).json({ message: 'Invalid Email or Password' });
        }
    } catch (error) {
        console.error('Login error:', error);
        res.status(500).json({ message: 'Server Error Occurred' });
    }
});
```

Use of app.get method in this project

The app.get() method is used to define a route handler for HTTP GET requests to a specific path (URL). To fetch data from the database and display inside the

project like if user login suceesfully in profile there should be display user details and also for history, etc..

```
// Student profile
app.get('/api/student/:email', async (req, res) => {

    try {
        const email = req.params.email;
        const user = await User.findOne({ email });
        if (!user) {
            return res.status(404).json({ message: 'User not found' });
        }
        res.json({
            Username: user.username,
            Email: user.email,
            Department: user.department,
            RollNumber: user.college_id,
            MobileNumber: user.mobileNumber || ''
        });
    } catch (error) {
        console.error('Profile fetch error:', error);
        res.status(500).json({ message: 'Server error' });
    }
});

// Teacher profile
app.get('/api/teacher/:email', async (req, res) => {
    try {
        const email = req.params.email;
        const user = await Teacher.findOne({ email });
        if (!user) {
            return res.status(404).json({ message: 'Teacher not found' }
        }
```

Use of app.put method in this project

In Node.js with Express, the app.put() method is used to define a route handler for HTTP PUT requests. PUT is an HTTP method used to update an existing resource on the server. Put method is used for users can update there profile and also used that in same in admin page also admin can update users details.

```
// Student update - existing
app.put('/api/student/update/:email', async (req, res) => {
    try {
        const email = req.params.email;
        const { Username, RollNumber, Department, MobileNumber } = req.body
        const updateData = {
            username: Username,
            college_id: RollNumber,
            department: Department,
            mobileNumber: MobileNumber
        };
        const user = await User.findOneAndUpdate(
            { email },
            { $set: updateData },
            { new: true, runValidators: true }
        );
        if (!user) {
            return res.status(404).json({ message: 'User not found' });
        }
        res.json({
            Username: user.username,
            Email: user.email,
            Department: user.department,
            RollNumber: user.college_id,
            MobileNumber: user.mobileNumber || ''
        });
    } catch (error) {
        console.error('Update error:', error);
        res.status(500).json({ message: 'Server error during update' });
    }
});
```

Nodemailer

Nodemailer is used send emails to the users while there are register and order any book are return any book and admin can send sms for return books to email, etc... for email send I create a separate file for email sending and connect to the server.js file.

In the below image we can see the connection to the registration in server.js file

```
app.post('/student_register', async (req, res) => {
    try {
        const { username, email, college_id, department, password, conformpassword } = req
        if (password !== conformpassword) {
            return res.status(400).json({ message: "Passwords do not match" });
        }
        const hashedpassword = await bcrypt.hash(password, 12);
        const newUser = new User({
            username, email, college_id, department, password: hashedpassword
        });
        await newUser.save();
        // Send welcome email
        try {
            await sendRegistrationEmail(newUser, 'student', department);
        } catch (emailErr) {
            console.error('Email failed but registration OK:', emailErr.message);
        }

        res.status(201).json({ message: "User registered successfully" });
        console.log("Student registered successfully");
    } catch (err) {
```

Email connection code for the registration in sendEmail.js

```
> JS sendEmail.js > sendReturnConfirmationEmail
const nodemailer = require('nodemailer');
require('dotenv').config();

const fromEmail = process.env.GMAIL_USER || 'kleelavinayak@gmail.com';

const createTransporter = () => {
    return nodemailer.createTransport({
        service: 'gmail',
        auth: {
            user: fromEmail,
            pass: process.env.GMAIL_PASS || 'ajdu qffu pacf esfq' // fallback for testing
        },
        tls: {
            rejectUnauthorized: false
        }
    });
};
```

```
const sendRegistrationEmail = async (user, role, department = '') => {
    const transport = createTransporter();

    const subject = `Welcome to Vemu Library - ${role.charAt(0).toUpperCase() + role.slice(1)}`;
    const text = department
        ? `Dear ${user.username}, you successfully registered as a ${role} in ${department} department.`
        : `Dear ${user.username}, you successfully registered as a ${role}.`;

    const mailOptions = {
        from: `"Vemu Library" <${fromEmail}>`,
        to: user.email,
        subject,
        text,
        html: `
            <h2>Welcome to Vemu Library!</h2>
            <p>Dear <strong>${user.username}</strong>,</p>
            <p>You have successfully registered as a <strong>${role.charAt(0).toUpperCase() + role.slice(1)
            <p>College ID: ${user.college_id}</p>
            <p>Email: ${user.email}</p>
            <p>Start exploring our library catalog!</p>
            <hr>
            <p>Best regards,<br>Vemu Library Team</p>
        `
    };
```

```
    try {
        await transport.sendMail(mailOptions);
        console.log(`✅ Registration email sent successfully to ${user.email} (${role})`);
        return true;
    } catch (error) {
        console.error(`❌ Failed to send email to ${user.email}:`, error);
        // Continue without throwing - registration succeeds
        return false;
    }
};

const sendFineAlertEmail = async (issue) => {
```

Nodemailer

Nodemailer is a powerful Node.js module for sending emails. Below is a step-by-step guide to configure and use Nodemailer in your Node.js application.

1. Install Nodemailer

Run the following command in your project directory to install Nodemailer:

```
PS C:\Users\leela\OneDrive\Desktop\Vemu library management system\backend> npm install nodemail
```

2. Create a Transporter

The transporter is responsible for handling the email delivery. Here's an example of configuring it with Gmail:

```
const createTransporter = () => {
    return nodemailer.createTransport({
        service: 'gmail',
        auth: {
            user: fromEmail,
            pass: process.env.GMAIL_PASS || 'ajdu qffu pacf esfq' // fallback for testing
        },
        tls: {
            rejectUnauthorized: false
        }
    });
};
```

3. Define Email Options

Specify the sender, recipient, subject, and content of the email:

```
const sendRegistrationEmail = async (user, role, department = '') => {
    const transport = createTransporter();

    const subject = `Welcome to Vemu Library - ${role.charAt(0).toUpperCase() + role.slice(1)}`;
    const text = department
        ? `Dear ${user.username}, you successfully registered as a ${role} in ${department} department.`
        : `Dear ${user.username}, you successfully registered as a ${role}.`;

    const mailOptions = {
        from: `"Vemu Library" <${fromEmail}>`,
        to: user.email,
        subject,
        text,
        html: `
            <h2>Welcome to Vemu Library!</h2>
            <p>Dear <strong>${user.username}</strong>,</p>
            <p>You have successfully registered as a <strong>${role.charAt(0).toUpperCase() + role.slice(
            <p>College ID: ${user.college_id}</p>
            <p>Email: ${user.email}</p>
            <p>Start exploring our library catalog!</p>
            <hr>
            <p>Best regards,<br>Vemu Library Team</p>
        `
    };
```

4. Send the Email

Use the sendMail method to send the email:

```
    try {
        await transport.sendMail(mailOptions);
        console.log(`✅ Registration email sent successfully to ${user.email} (${role})`);
        return true;
    } catch (error) {
        console.error(`❌ Failed to send email to ${user.email}:`, error);
        // Continue without throwing - registration succeeds
        return false;
    }
};
```

This is about the backend which is used inside in this web application.

Frontend

Frontend (or client-side) development is everything users see and interact with in a web application or website. It runs inside a web browser (Chrome, Firefox, Safari, Edge) and focuses on:

- Layout & structure
- Visual styling
- User interactions
- Performance & accessibility

Core Technologies

Technology	Purpose
HTML	Structure/content (headings, paragraphs, images, forms)
CSS	Styling (colors, fonts, spacing, animations, layout)
JavaScript	Interactivity (clicks, form validation, API calls, dynamic updates)

JavaScript Frameworks & Libraries

Library/Framework	Description
React.js	Most popular, component-based, virtual DOM, by Meta

What is React?

React is a **JavaScript library** for building user interfaces, especially single-page applications (SPAs). It lets you build reusable UI components that manage their own state.

Core Concepts

Concept	Explanation
Components	Reusable, isolated pieces of UI (function or class)

Concept	Explanation
JSX	JavaScript + XML syntax — write HTML inside JavaScript
Props	Read-only data passed from parent to child
State	Mutable data that triggers re-renders when changed
Hooks	Functions that let you use state & lifecycle in functional components (e.g., useState, useEffect)
Virtual DOM	Lightweight copy of real DOM for efficient updates
One-way data binding	Data flows from parent to child via props

Example: Simple React Component

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

export default Counter;
```

User Roles & Permissions

Role	Access Level
Student	Order books, view borrowed books, pay/view fines (₹10/day late fee).
Teacher	Order books, borrow with no fines, flexible return dates.
Librarian	Add books, mark books as returned, view return history.
Admin	View all booking history, send SMS reminders, update user details, approve new

Student Dashboard (/student)

Features:

- Browse Books: Search by title, author, or category.
- Place Order: Click “Borrow” on available book (max limit: 3 books).
- My Borrowings: Table showing:
    - Book name, Borrow date, Due date (15 days from borrow).
    - Fine column: Calculated automatically if returned late (₹10/day).
    - Return button (available until returned).
- Fine Summary: Total pending fine.
- Notifications: Alert if book is overdue.

Teacher Dashboard (/teacher)

Features:

- Browse & Order Books: Same as student but no due date enforcement.
- My Borrowings: Shows borrowed books but no fine column.
- Return button: Works same as student.
- Message: “No fines applicable for faculty.”

3.4 Librarian Dashboard (/librarian)

Features:

- Add New Book: Form with – Title, Author, ISBN, Total Copies, Category, Shelf No.
- Return Book:
    - Search by Student/Teacher name or Book ID.
    - Click “Mark as Returned” → Updates inventory + removes from user’s borrow list.
- Return History Log: Table showing – User name, Book, Return date/time, Role (Student/Teacher).
- View Active Borrowings: See who currently has which book.

Admin Dashboard (/admin)

Tabs/Sections:

A. Pending Registrations

- List of new users (Name, Role, Email).
- Buttons: Approve (adds to system) / Reject.

B. User Management

- Search & update any user’s details (Name, Phone, Role, Department).
- Disable/Enable user accounts.

C. Booking History (All Users)

- Filter by: Role (Student/Teacher), Date range, Book name.
- Columns: User, Book, Borrow date, Return date, Fine amount (if any).

D. Fine Management & SMS

- Show students with unpaid fines.
- Send SMS Button: Predefined message – *“Dear [Name], please pay ₹[Fine] and return book soon.”*
- (SMS via backend API integration).

Middleware Connection

Middleware serves as a crucial intermediary that facilitates communication and data exchange between different applications, systems, or services, enabling se amless integration and interaction.

```
const Studentregister = async (e) => {
  e.preventDefault();
  const { username, email, college_id, depart const department: string word } = studentRegister;
  if (!username || !email || !college_id || !department || !password || !conformpassword) {
    showPopup("Enter the below details", 'warning');
    return;
  } else if (password !== conformpassword) {
    showPopup("Conform Password doesn't Matched", 'alert');
    return;
  }
  try {
    const response = await fetch('http://localhost:3000/student_register', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ username, email, college_id, department, password, conformpassword }),
    });
    const data = await response.json();
    if (response.ok) {
      showPopup(`${data.message} Check your email for welcome details!`);
      // after registration also store email and role so edit page can load
      localStorage.setItem('Email', email);
      localStorage.setItem('Role', 'student');
      localStorage.setItem('role', 'student');
      navigate('/student');
    } else {
      showPopup(data.message);
    }
```

```
    // response.status(200) = navigate('/student');
  } catch (error) {
    showPopup('Error registering. Please try again.', 'failed');
    console.log(error);
  }
}
```

This images used to show how I connect the frontend and backend. Every page should be contain same code to connect but API should be different.